

ConstructionSceneReady: Multi-scale IFC-to-OpenUSD compilation and scene-level validation for humanoid robotic assembly

Abstract

Robot-ready construction simulation requires more than converting building geometry into a simulation format. A composed scene must preserve engineering identity, expose assembly interfaces and construction states, allocate physical fidelity at the scale required by the active task, and close all references among the building, robot, tool, and task. Existing simulation-readiness specifications primarily validate individual assets, leaving scene-level construction task readiness to expensive runtime discovery. This study proposes ConstructionSceneReady, a multi-scale IFC-to-OpenUSD compiler and scene-level validation framework for humanoid robotic assembly. It represents construction-task readiness as an explicit contract and compiles the IFC source into global-building, active-work-zone, and interaction-level USD layers. Critical physical properties retain their provenance and uncertainty. When deterministic validation fails, a language-model agent may diagnose the structured report and invoke only approved engineering repair tools. A fault-injection benchmark and paired humanoid assembly simulations compare the framework with direct conversion, fixed-rule conversion, and unconstrained agentic repair. On a frozen 105-case fault-injection partition the deterministic validator recovers all 141 expected findings without false positives and repairs 97.1% of cases to pristine-scene equivalence, escalating only cases whose repair would require unsupported physical values; a validator-grounded agent matches this ceiling (96.8% valid, zero incorrect repairs) while an unconstrained agent repairs only 32.8% and returns validator-passing but incorrect scenes in 22.7% of 525 episodes. In 5,600 paired humanoid insertion episodes, symbolic and referential scene defects were discretely fatal, assembly interface clearance was the most sensitive continuous feature, and absolute member-pose errors were absorbed by closed-loop perception. The framework is intended to make scene-model failures observable before they become costly robot-runtime failures.

Keywords: building information modeling, OpenUSD, construction robotics, humanoid robot, simulation validation, agentic repair

1. Introduction

Robotic construction is increasingly planned from digital building information, yet the model delivered by a designer and the world required by a robot remain fundamentally different artefacts. An Industry Foundation Classes (IFC) model describes intended components, spatial containment, materials, and design relationships. A robot simulator instead requires collision geometry, mass properties, physical constraints, task targets, and an explicit account of which objects and geometric details are active. The difference is not resolved by changing file formats. It is a change from a representation of what should be built to a representation of what can be acted upon, under a particular construction state and by a particular embodiment.

The problem becomes more pronounced at building scale. Navigation through a floor or room can be evaluated with simplified walls, openings, and obstacles, whereas placing a connection plate or inserting a pin depends on millimetre-scale interface geometry. Loading every building object at interaction-level fidelity is wasteful and can make collision checking and physics simulation unnecessarily expensive. Retaining only coarse geometry is equally problematic: it can conceal penetrations, reverse an insertion axis, or turn an infeasible assembly into an apparently valid one. A useful robot world must therefore change its working set and geometric fidelity with the task without losing identity or relationships across scales.

Previous studies have established impor-

tant parts of this information chain. BIM-derived robot worlds and task-planning systems have been generated for painting, prefabricated assembly, and interior wall installation [1–3]. IFC semantics have also been linked to action-level robot control through the Robot Construction Action Node [4], while closed-loop digital twins have used as-built observations and human intervention to adjust BIM-driven construction workflows [5]. These studies show that building information can drive robot planning and simulation. They do not, however, establish a general criterion for deciding whether a composed construction scene is ready for a specified robot task before that task is executed.

In parallel, physics-ready asset generation has become considerably more automated. Xu et al. converted construction-equipment CAD data and mechanical specifications into articulated USD models, using a large language model to recover missing joint information [6]. OpenUSD now provides a strong technical basis for composing such assets through layers, references, and payloads [7]. Ren et al. have already shown that information delivery requirements can drive a specification-based IFC-to-USD pipeline while preserving lifecycle identifiers and model structure [8]. NVIDIA’s SimReady framework further organizes machine-checkable asset requirements into profiles, features, and rules [9]. This progress is essential, but an individually valid beam, tool, robot, or collider does not imply a valid assembly scene. The assets may refer to inconsistent coordinate frames, a task may address the wrong IFC object, the re-

quired work-zone payload may be absent, or the construction state may violate an action precondition. The current SimReady rules operate at asset and feature level; the official documentation notes that scene-level specifications and validators are not yet defined [10].

Figure 1 summarizes the resulting transition. Asset-oriented checks can establish that individual models contain valid geometry, collision shapes, or physical attributes, but the construction task is evaluated over a composition. Readiness therefore depends on closed cross-asset references, consistent spatial frames, the active work-zone payload, construction-state preconditions, and an interaction model whose fidelity matches the intended action.

Construction makes this scene-level gap particularly consequential. Temporary states such as pre-positioned, aligned, partially inserted, and temporarily fixed are not normally represented in an as-designed model, although they determine whether the next robotic action is safe and meaningful. Physical properties may also be assembled from sources of unequal authority: geometry and identity may be explicit in IFC, mass may be calculated from volume and material density, friction may come from a published interval, and an assembly tolerance may be introduced as a task-specific assumption. If these values are stored without provenance, a simulator can produce a precise-looking result whose physical basis cannot be audited. A language-model agent compounds the problem when it is allowed to invent or modify engineering parameters without a deterministic acceptance

test.

Humanoid robots provide a demanding but useful setting in which to examine these issues. Human-oriented buildings contain doors, thresholds, narrow work zones, tools, and operating heights that motivate legged mobility and whole-body manipulation. This does not make a humanoid universally preferable to a mobile manipulator; wheeled systems retain clear advantages in payload, stability, and operation on regular floors. Rather, a humanoid task exposes the need to connect building-scale circulation, body placement, bimanual reach, and local contact geometry in one composed scene. The present work therefore uses light-steel assembly by a Unitree G1-class humanoid as the evaluation context, while keeping the proposed scene representation independent of a particular controller. A recent construction-humanoid digital-twin study has already demonstrated the conversion of BIM geometry and semantic information into a MuJoCo scene for Unitree G1 task decomposition and simulation [11]. The question pursued here is consequently not whether BIM and a humanoid can coexist in a simulator, but whether the resulting multi-asset scene can be compiled, audited, repaired, and related quantitatively to task outcome.

This paper develops Construction-SceneReady around the proposition that construction scene readiness should be compiled and tested explicitly, rather than discovered only through failed simulation runs. IFC4.3 entities are first translated into a construction scene intermediate representation that retains component identity,

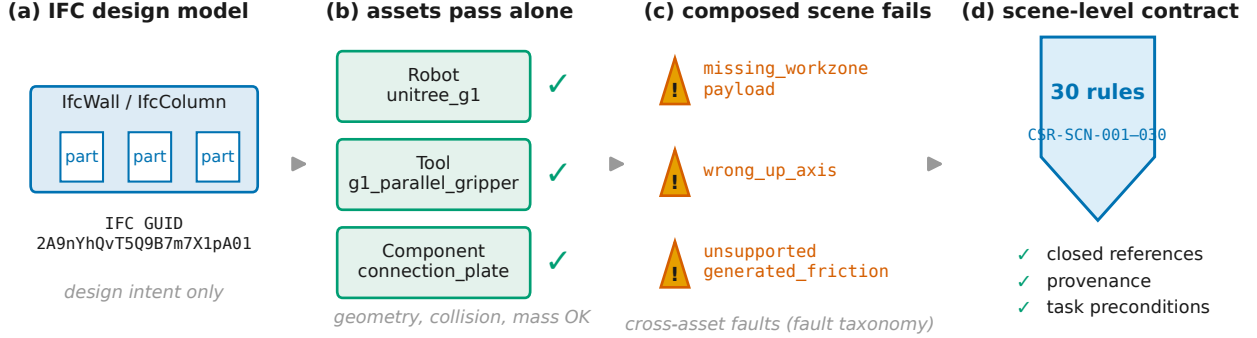


Figure 1: From individually valid digital assets to a task-ready construction scene. Scene-level failures emerge from composition, construction state, and task binding even when each asset passes its own checks.

work zones, assembly interfaces, construction states, tasks, and property provenance. The representation is then compiled into a global navigation layer, one or more work-zone payloads, and task-specific interaction layers. This organization permits OpenUSD to load detailed geometry only where the active task requires it, while references preserve the connection to the same engineering objects across layers.

A deterministic contract is evaluated over the composed scene rather than over isolated assets. Its rules cover coordinate and unit consistency, reference closure, task-to-interface binding, state preconditions, work-zone activation, and the availability and provenance of critical physical parameters. When a rule fails, an agent receives a structured diagnostic and may select only from a whitelist of repair operations, such as restoring an IFC-USD mapping, recomputing inertia, or regenerating an allowed payload. The validator, rather than the agent, decides whether the revised scene is acceptable. Pa-

rameters whose authority or risk exceeds a defined threshold are escalated for human review.

The evaluation is built around controlled failure rather than a single successful demonstration. Parameterized IFC4.3 fixtures generate correct scene descriptions and machine-readable ground truth for connection-plate positioning, pin insertion, and sequential assembly. Known semantic, spatial, physical, interface, state, and task defects are then injected to compare generic asset checks, fixed-rule conversion, unconstrained agentic repair, and validator-grounded repair. CPU experiments measure detection, repair reliability, provenance coverage, and the resource effect of task-activated layers. Subsequent RTX experiments test whether the resulting readiness scores and critical rule failures explain motion-planning and humanoid assembly outcomes. In this way, the paper links an information-model claim—that a scene satisfies an explicit construction contract—to an operational claim—that the composed world

supports the intended robotic task.

2. Related work

2.1. Construction robotics and the role of building information

Construction robotics has long been framed as an information problem as much as a machine problem. Early surveys anticipated that sensing and product models would be needed to compensate for site variability [12–14]. Later reviews identified persistent barriers in localization, work preparation, safety, interoperability, and adaptation to unstructured conditions [15–17]. The diversity of the field is relevant to scene modelling: collective fabrication, on-site mobile systems, prefabrication, and human–robot collaboration impose different requirements on the digital representation used at runtime [18–20]. Consequently, “robot-ready” cannot denote a single geometric level of detail or one fixed set of asset attributes.

Reviews focused specifically on BIM–robot interaction show a progression from one-way export towards lifecycle information flow and closed-loop exchange [21–24]. Across this literature, BIM supplies geometry, object identity, material and spatial semantics, schedules, and task constraints. The receiving robotic system commonly creates its own reduced representation for a particular purpose. A navigation map, an assembly graph, an SDF world, and an articulated physics asset are all robot-relevant, but they make different claims about what information is complete and valid. This fragmentation motivates the

present emphasis on an explicit scene contract rather than another destination format.

For robotic assembly, Dawod and Hanna used BIM geometry and design context to support object recognition, grasp search, and motion planning for discrete structures [25]. Ding et al. connected image-based reconstruction to BIM task-level planning for brick assembly [26]. Wong Chong et al. automated the generation of robot simulation input for timber-frame assembly [27], while Gao et al. combined BIM-based task and motion planning for lightweight emergency structures [28]. These studies established that building objects can be transformed into executable task information; they did not seek to define a common acceptance test for the composed simulator stage.

Kim et al. developed a BIM-integrated planning and simulation system in which building information was converted to an SDF world and used to configure robot painting tasks [1]. Zhu et al. subsequently proposed an open, component-oriented pipeline that split IFC building elements, preserved their identity, converted them into SDF smart components, and executed a light-steel assembly sequence in ROS/Gazebo [2]. This work established automated IFC-to-simulator conversion as feasible, but assembly interface validity and cross-component physical consistency remained outside its scope.

Research then moved closer to action-level construction information. The RCAN framework linked IFC-derived data to ordered construction actions and ROS execution, enabling changes in component-oriented

plans to be propagated to a mobile manipulator [4]. Oyediran et al. integrated 4D BIM, a construction-robot knowledge base, high-level task planning, and action-level simulation for indoor wall-frame installation [3]. These systems enrich BIM with task knowledge and make robot operations more context-aware. Their principal concern is the creation and flow of planning information, rather than a formal test of whether all assets, references, intermediate states, and physical assumptions form a valid task scene.

Semantic world modelling provides a second route from BIM to robot execution. Kim and Peavy represented building elements, spaces, and robot-relevant relationships in a semantic building world [29]. For mobile robots, IFC has been converted into maps, waypoints, and schedule-aware Gazebo environments [30, 31]. Knowledge-driven inspection and progress-monitoring systems likewise use BIM to select targets and interpret observations [32, 33]. Ontologies have been proposed to regularize the vocabulary for navigation and inspection [34], and semantic BIM attributes have been used to alter path cost and clearance in safety-aware planning [35]. This line of work demonstrates the operational value of semantics, but its main unit of analysis is a map, waypoint, or knowledge base. Assembly readiness additionally requires valid contact interfaces, physical parameters, object states, and task success conditions.

2.2. Physics-ready assets and multi-scale scene composition

Robotic simulation depends on asset properties that BIM models rarely provide in executable form. Xu et al. addressed this issue for construction equipment by fusing CAD geometry, equipment templates, and technical specifications into articulated USD models [6]. Their method generated hierarchy, joints, drives, materials, and collision representations for a wheel loader, and used a large language model to extract unspecified mechanics. The study substantially reduced modelling time and is a close precedent for automated physics-ready generation. Its stated scope was equipment modelling rather than environment modelling or equipment–environment interaction. It therefore does not resolve the composition of building components, temporary states, task interfaces, and robot capabilities into one assembly scene.

General-purpose robotics simulators make the representational gap visible in different ways. MuJoCo emphasizes efficient rigid-body dynamics and contact-rich model-based control [36]; GPU-native simulation environments such as Isaac Gym make it practical to execute many robot worlds in parallel [37]. These runtime capabilities do not establish that a source model was compiled correctly. Domain randomization can expose policies to variations in visual or physical parameters [38], but it presupposes a valid nominal model and defensible parameter ranges. It is therefore distinct from detecting a reversed coordinate frame, a missing assembly mate, or a physical value with no authoritative source.

OpenUSD is well suited to scene composition because its layers, references, variants, and payloads separate asset definitions from scene opinions and permit data to be loaded on demand [7]. This is important for construction: a building shell may support navigation at coarse resolution while an active connection requires detailed collision and contact geometry. Ren et al. developed a specification-driven IFC-to-USD pipeline in which information delivery requirements act as a scene contract for selecting and mapping identifiers, hierarchy, geometry, materials, and attributes [8]. Their study establishes both lifecycle-oriented semantic preservation and configurable IFC-to-USD delivery; neither can be claimed as new here. What remains unresolved for robot construction is how a current task should activate geometric scales, physical interfaces, temporary states, and cross-asset acceptance conditions. IfcOpenShell provides access to IFC4.3 objects, property sets, spatial containment, and geometry [39, 40], but a domain-specific compilation policy is still required to turn that information into an auditable task scene.

SimReady complements USD composition with machine-readable capabilities, features, profiles, and validation rules [9]. Current profiles are valuable for checking individual assets, including rigid-body, collider, material, hierarchy, and robot-articulation requirements. An asset-level result cannot establish that a particular robot, tool, work zone, component, and target interface have been composed consistently. The official FAQ identifies scene-level specifications and validators as a limitation of the current frame-

work [10]. ConstructionSceneReady therefore builds on, rather than replaces, asset validation by adding relations that become meaningful only after a construction task scene is composed.

Scene generation for embodied AI is advancing in parallel and narrows the claim space further. SimRecon reconstructs compositional simulator scenes from video through perception, object generation, and scene-graph-guided assembly [41]. EmbodiedGen V2 generates editable, task-driven worlds with affordances and cross-simulator deployment, and evaluates whether the worlds support downstream policy learning [42]. Agentic Real2Sim recovers geometry, physical parameters, robot interaction, and object state from recorded episodes using vision-language agents [43]. These systems show that agentic production of runnable, physically enriched scenes is no longer an unoccupied research problem. Their source of authority is visual observation or generative world specification, whereas a construction scene must also remain accountable to IFC identity, engineering interfaces, process state, and explicitly bounded parameter sources. This difference motivates an auditable compiler and fault benchmark rather than another general-purpose scene generator.

The use of a humanoid evaluation platform makes this cross-scale requirement especially visible. Whole-body benchmarks show that locomotion and manipulation outcomes depend on coupling among support, reach, contact, and object dynamics [44]. In a construction scene, the coupling extends across building circulation, body placement,

work-zone occupancy, and millimetre-scale assembly geometry. A compiler must support coarse global context and detailed active interfaces without representing every building object at maximum fidelity. This is a scene-composition requirement, not a claim that humanoids are universally preferable to wheeled mobile manipulators. Nor is BIM-to-humanoid simulation itself new: Ye et al. convert BIM geometry and semantic information to a MuJoCo scene and demonstrate task decomposition with Unitree G1 [11]. The present use of G1 is therefore an evaluation stress test for task-scene readiness rather than a standalone contribution.

2.3. Closed-loop and robot-ready construction twins

Digital-twin research has addressed the discrepancy between design models and changing construction sites. Boje et al. argued for semantically structured construction twins that connect heterogeneous representations and lifecycle processes [45]. Sacks et al. placed the twin within a closed-loop production system in which observations and decisions continually update construction control [46]. These formulations show that a design model is only one view of a changing physical system. They do not, however, prescribe a simulator-specific readiness contract for a robotic task.

Wang et al. proposed a closed-loop BIM-driven workflow in which a human could revise task sequences, target positions, and motion plans when drywall installation encountered as-designed/as-built differences [5].

Tuomisto et al. used BIM to initialize LiDAR SLAM and next-best-view planning for quadruped inspection, while allowing the map to distinguish designed structures, missing elements, and dynamic obstacles [47]. These approaches make robot operation robust to observed site conditions, but they validate the world through runtime sensing and interaction rather than through pre-runtime analysis of a composed simulation scene.

The BIM2RDT work of Akhavian et al. further integrates BIM geometry, robot observations, IoT data, semantic registration, and safety-aware navigation [48]. It represents an important step toward dynamic, robot-ready construction twins and LLM-orchestrated workflows. Its principal technical contributions concern point-cloud registration, perception, monitoring, and navigation constraints. The present work addresses a complementary stage: compiling an auditable physical scene before simulation, then determining whether its cross-asset relationships and construction-task assumptions are internally ready for execution.

Engineering analysis can also serve as an acceptance gate. The integration of BIM, finite-element analysis, and ROS has shown how temporary structural stability can modify a robot assembly plan [49]. BIM-semantic-web integration can supplement an observed component with design relationships and requirements [50]. These studies demonstrate that readiness is conditional on the current state and action, not an intrinsic label attached to an asset. ConstructionSceneReady adopts that conditional view while limiting its present physical scope to rigid components

and simplified contact interfaces.

2.4. Language models, agents, and engineering assurance

Language models have begun to support construction-robot planning and engineering-model generation. RoboGPT used ChatGPT to generate and revise assembly sequences, demonstrating that natural-language reasoning can adapt a robot plan across construction cases [51]. Xu et al. used an LLM more narrowly to extract equipment-joint parameters from technical documents [6]; BIM2RDT uses language-model reasoning to assign semantic priors and orchestrate parts of a robot-ready twin pipeline [48]. These studies show the value of models that connect heterogeneous descriptions and select procedural steps.

The wider robotics literature offers several patterns for grounding language in executable behavior. SayCan combines language-model preferences with learned affordance values [52]; Inner Monologue uses environment feedback to revise embodied plans [53]. Code as Policies and Program Prompt constrain generated output through executable program structure or situated action specifications [54, 55]. Multimodal embodied models such as PaLM-E and vision-language-action models such as RT-2 move perception and action closer to the language model itself [56, 57]. Multi-robot planning frameworks further decompose tasks, allocate capabilities, and produce programmatic plans [58]. These methods strengthen task generation, but none of them makes a language model an authoritative source for

an engineering mass, tolerance, coordinate frame, or IFC identity.

A plausible action sequence is not necessarily executable, and a plausible numerical parameter is not necessarily authoritative. In ConstructionSceneReady, the agent is not treated as a validator or a source of ground truth. It receives a typed error report, chooses from bounded repair tools, records the evidence for each mutation, and returns the scene to deterministic rules. This pattern differs from direct language-to-scene generation because acceptance remains independent of the model that proposed the repair. It also makes fixed-rule and agentic methods comparable on identical injected faults.

The separation of proposal from acceptance is consistent with hybrid planning in which a language model supplies a rough plan and a formal answer-set planner establishes executability [59]. It also aligns with fault-oriented robot assurance: injected faults can determine whether a digital representation and its deployment process expose unsafe assumptions before hardware execution [60]. In the present benchmark, the object under test is the construction scene. The agent may select a repair, but typed rules, source authority, and task-level checks determine whether that repair is accepted.

2.5. Synthesis of the research gap

Table 1 compares the closest methodological precedents. The reviewed literature establishes IFC-to-simulator conversion, BIM-derived robot tasks, semantic maps, specification-driven IFC-to-USD delivery, physics-ready USD asset generation,

agentic simulation-world generation, BIM-to-
 humanoid simulation, closed-loop twin up-
 dating, semantic perception, and language-
 model-assisted planning. These capabilities
 are necessary parts of a robot-ready workflow,
 but they operate on different units of analysis.
 Conversion studies focus on delivered build-
 ing information, physics-ready studies on an
 equipment asset, generative-world studies on
 executable scenes, planning studies on action
 information, and digital-twin studies on run-
 time alignment with a site.

The literature search did not identify a
 study that jointly evaluates a construction
 task scene contract, task-activated multi-
 scale USD composition, physical-property
 provenance, deterministic scene-level fault de-
 tection, validator-grounded repair, and the
 downstream outcome of a humanoid assem-
 bly task. This is deliberately narrower than
 a claim of first IFC-robot interoperability or
 first USD validation; both are established ar-
 eas. The unresolved transition is from indi-
 vidually valid digital assets to an auditable,
 composed, task-specific construction world.

Two buildingSMART instruments deserve
 explicit positioning. The Information Deliv-
 ery Specification (IDS) [61] provides machine-
 checkable information requirements for IFC
 deliveries, and the buildingSMART Data Dic-
 tionary (bSDD) [62] standardizes the vocabu-
 lary those requirements reference. Both oper-
 ate at the level of individual entities and their
 declared properties: an IDS facet can require
 that a plate carries a fire rating, but it can-
 not express that a task target must resolve
 to a component that owns the bound inter-
 face, that an active work zone’s payload must

stay inside the scene package, or that a crit-
 ical physical value must carry an authorita-
 tive provenance record. The present contract
 is complementary by design: bSDD-style dic-
 tionaries supply standardized semantics, IDS-
 style facets supply per-entity requirements,
 and the scene contract adds cross-reference
 closure, task binding, composition safety, and
 evidence authority at the level of the com-
 posed scene. Section 7 further shows em-
 pirically that even a fully satisfied contract
 leaves sub-tolerance geometric fidelity invis-
 ible, which motivates the candidate rules reg-
 istered there.

Table 1: Capabilities of closely related BIM–robotics and physics-ready modelling studies. A filled cell indicates that the capability was a substantive part of the reported method or evaluation.

Study	BIM/IFC	Robot task	Physical asset	Multi-scale	Scene rules	Provenance	Agent repair	Runtime outcome
Kim et al. [1]	✓	✓	partial	–	–	–	–	✓
Wong Chong et al. [27]	✓	✓	partial	–	–	–	–	✓
Zhu et al. [2]	✓	✓	partial	–	–	–	–	✓
Oyediran et al. [3]	✓	✓	partial	–	–	–	–	✓
Xu et al. [6]	–	–	✓	–	asset	partial	partial	✓
Ren et al. [8]	✓	–	partial	scene	specification	✓	–	–
Wang et al. [5]	✓	✓	partial	–	runtime	–	human	✓
Gopee et al. [31]	✓	navigation	partial	map	runtime	–	–	✓
Akhavian et al. [48]	✓	navigation	partial	–	runtime	partial	partial	✓
Ye et al. [11]	✓	✓	partial	scene	–	–	–	✓
Wang et al. [42]	–	✓	✓	✓	partial	partial	generation	✓
ConstructionSceneReady	✓	✓	✓	✓	scene	✓	✓	✓

3. Problem definition

Let I be an IFC model, K a set of construction-domain rules, R a robot description, and T a task specification. The compiler produces an OpenUSD scene S and a provenance ledger P :

$$C(I, K, R, T) \rightarrow (S, P). \quad (1)$$

The scene is composed from three fidelity layers:

$$S = S_{\text{global}} \oplus S_{\text{workzone}} \oplus S_{\text{interaction}}, \quad (2)$$

where $\oplus$ denotes OpenUSD composition. The active task determines which work-zone and interaction payloads are loaded.

A construction scene contract is a versioned set of requirements $\mathcal{Q} = \{q_1, \dots, q_n\}$. A deterministic validator returns:

$$V(S, T, \mathcal{Q}) \rightarrow E, \quad (3)$$

where E is a structured set of violations containing a rule identifier, severity, scene path, evidence, and allowed repair tools.

For every critical generated property p_i , the compiler stores:

$$p_i = (v_i, s_i, c_i, u_i), \quad (4)$$

where v_i is the value, s_i its source, $c_i \in [0, 1]$ a confidence score, and u_i an uncertainty description or interval.

The repair policy chooses only from a whitelist $\mathcal{A}$:

$$a^* = \arg \max_{a \in \mathcal{A}} P(V(a(S), T, \mathcal{Q}) = \emptyset \mid E, P), \quad (5)$$

subject to a risk rule that escalates safety-critical or unsupported mutations to human review.

4. Methodology

4.1. Construction Embodied World Graph

The intermediate representation is implemented as a Construction Embodied World Graph (CEWG). Its typed nodes contain components, assembly interfaces, construction states, robots, tools, tasks, work zones, robot capabilities, and provenance records.

Edges make cross-entity claims explicit, including `located_in`, `has_interface`, `mates_with`, `performed_by`, `uses_tool`, `targets`, and `can_transition_to`. It is deliberately simulator-neutral so that IFC extraction, graph-integrity checks, USD authoring, scene validation, and runtime adapters remain separately testable.

Figure 2 shows how these modules form one auditable compilation pipeline. Data artefacts are identified on the arrows so that implementation outputs, validation evidence, and runtime measurements can be traced back to their source.

The typed entities and relations are specified in Figure 3. The figure also exposes provenance as part of the representation rather than as an external note attached after simulation.

4.2. Task-activated multi-scale compilation

The compiler maps building circulation and architectural boundaries to a coarse global layer. It emits each work zone as an independently loadable payload and reserves interface-level geometry, tolerances, contact properties, and temporary constraints for the active interaction layer. IFC GUIDs remain stable across all layers.

The intended OpenUSD composition is illustrated in Figure 4. The final inset will report the measured working-set effect; it will not be populated until the RTX batch experiment has been executed.

4.3. Construction scene readiness contract

Rules are grouped into identity and reference closure, coordinate and unit consistency,

interface and tolerance completeness, state-transition validity, robot capability and task preconditions, physical stability, provenance, and multi-scale fidelity. Each rule is deterministic and emits a machine-readable failure.

Table 2 freezes the contract categories and the evidence required from the implementation. The full list of versioned rules and unit tests will be released as supplementary material rather than compressed into the main text.

4.4. Validator-grounded repair

The repair agent receives no free-form authority over the scene. It may inspect evidence, select a permitted tool, propose bounded parameters, execute the tool, and request re-validation. Unsupported or high-risk changes are escalated.

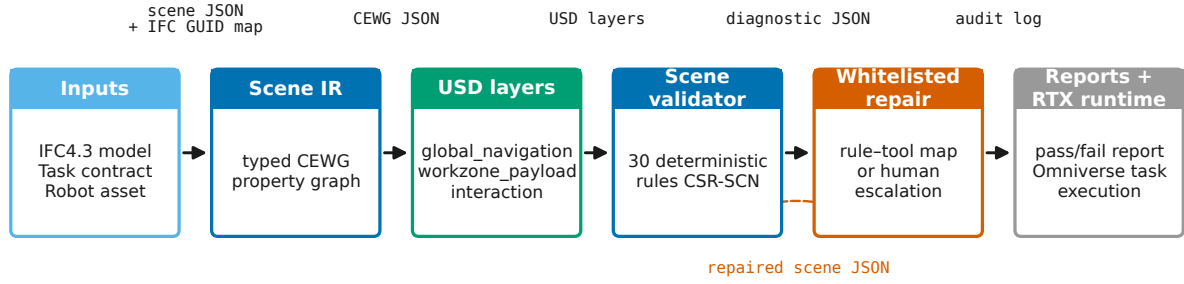


Figure 2: Architecture of ConstructionSceneReady. Deterministic compilation and validation remain separate from agentic diagnosis and from the downstream simulation runtime.

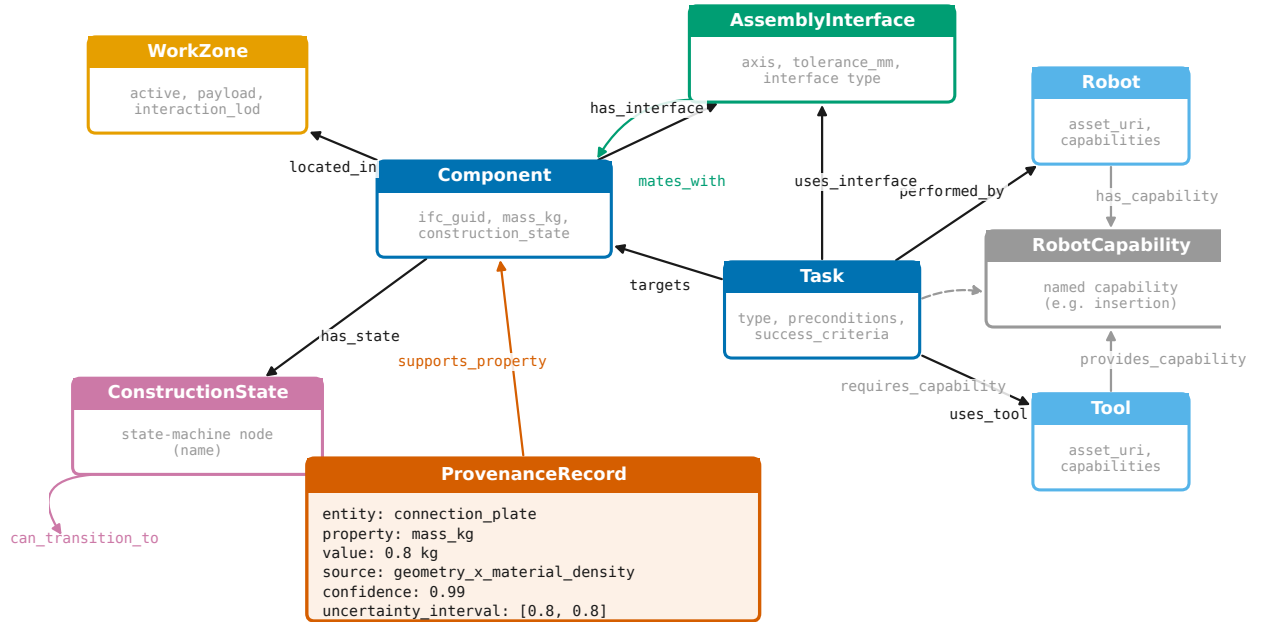


Figure 3: Construction Embodied World Graph and property-level provenance. The same engineering identity connects source IFC entities, composed USD prims, task targets, and validation evidence.

OpenUSD composition tree — pin_insertion (task_activated mode)

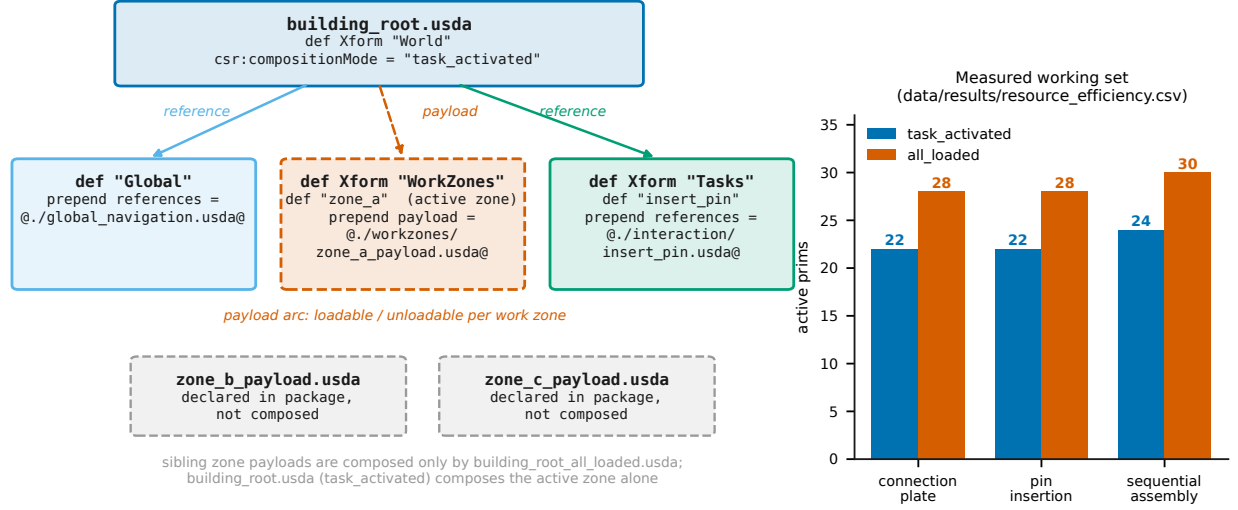
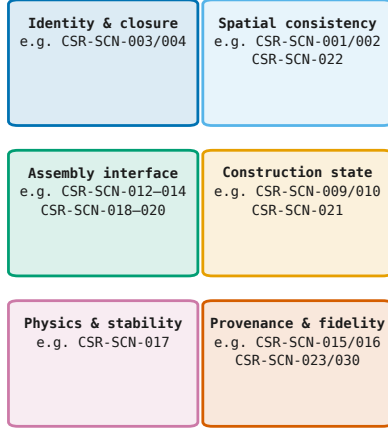


Figure 4: Task-activated multi-scale scene composition. Fine contact geometry and temporary constraints are loaded only for the active interaction while engineering identity is preserved across layers.

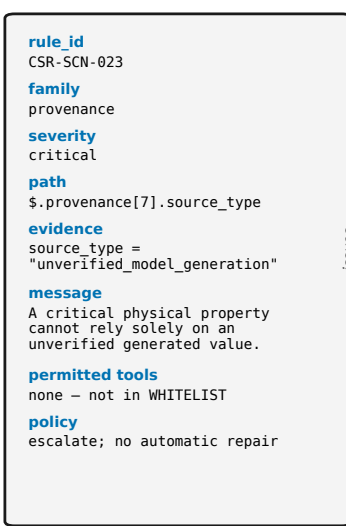
Contract families

(docs/scene-readiness-rules.csv)



30 deterministic rules; every violation emits a
typed, machine-readable diagnostic

Diagnostic record (true validator output)



Repair state machine

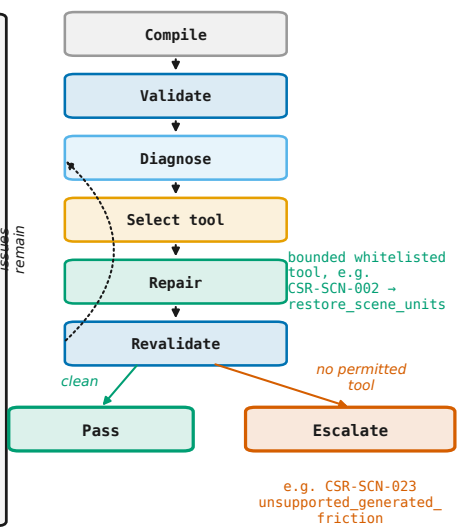


Figure 5: Validator-grounded repair. The agent interprets structured evidence and selects a bounded operation, whereas deterministic rules decide acceptance and high-risk or unsupported mutations are escalated.

Table 2: Construction-scene readiness contract to be implemented and evaluated. The final manuscript will report versioned rule identifiers and coverage in the supplementary material.

Contract category	Required scene evidence	Deterministic check	Severity	Consequence if violated
Identity and closure	IFC GUID, USD prim path, closed task and asset references	Uniqueness and reference resolution	Critical	Wrong or unresolved task object
Spatial consistency	Units, up axis, frames, transform chain, work-zone membership	Transform normalization and bounds	Critical	Unreachable or misplaced interaction
Assembly interface	Paired interface, insertion axis, tolerance, grasp and support regions	Geometric and relational constraints	Critical	False alignment or infeasible insertion
Construction state	Current state, legal predecessor, action precondition, completion criterion	State-machine validation	Critical	Unsafe or meaningless action
Physics and stability	Mass, inertia, collision, contact property, temporary support	Physical admissibility and support tests	Critical or major	Unstable or non-physical simulation
Provenance and fidelity	Property source, confidence, uncertainty, activated payload and level of detail	Authority and task-fidelity policy	Major	Unauditable result or hidden interaction error

5. System implementation

The CPU-first implementation uses Python for the intermediate representation, validation, fault injection, and experiment⁷⁵⁰ orchestration. IfcOpenShell is used for IFC extraction [40]; the OpenUSD Python API authors composable scene layers [7]. Runtime adapters are isolated from the core package so that the same compiler and val-⁷⁵⁵idators run on macOS development machines and Ubuntu RTX nodes.

The current reference implementation gener-⁷²⁵ates three IFC4.3 fixtures: connection-plate positioning, pin insertion, and sequential assembly. Each IFC file is accompanied by machine-readable ground truth and is compiled into an intermediate scene document and a canonical CEWG JSON document. A published JSON Schema constrains its node and edge types, while an independent integrity check rejects duplicate nodes and dangling graph relations. The fixture generator⁷³⁰ also supports paired deterministic variants of component scale, three-dimensional placement offset, and interface tolerance. Low-

discrepancy factor sequences are used so that the benchmark can be regenerated without a hidden random state; component mass changes with the cube of geometric scale to preserve material density.

IFC placements are multiplied by the file's declared length-unit scale before being written to the meter-normalized scene. This step is tested against generated ground truth because an IFC file expressed internally in millimetres can otherwise produce a syntactically valid USD scene whose object translations are wrong by three orders of magnitude. The USD output consists of a global navigation layer, independently loadable work-zone payloads, task-specific interaction layers, and a root stage that composes them through references and payloads. Thirty versioned rule identifiers currently cover coordinate normalization, IFC identity, work-zone activation, task references and contracts, robot and tool capabilities, assembly-interface closure, safe payload resolution, and critical-property provenance.

The local benchmark currently implements 30 single-fault operators and five compound faults spanning spatial frames, identity, layer composition, task contracts, embodiment, interfaces, physical validity, construction state, and provenance. Applied to the three fixtures, they produce 105 controlled cases. A strict rule-to-tool registry contains 29 bounded repair operations. One challenge fault, an unsupported model-generated friction value, is intentionally excluded from automatic repair and must be escalated. Every mutation records its rule, tool, changed scene paths, and trusted evidence before the

complete validator is rerun. These CPU cases verify the mechanics and measurement code; they do not replace the frozen baseline comparison or the downstream RTX task experiment. The complete implementation, frozen partition manifest, experiment registry, environment lock files, and per-episode ledgers are organized for release; the permanent repository DOI will be inserted at acceptance (see Data and code availability).

6. Experimental design

6.1. Scenarios and tasks

Three parameterized light-steel assembly scenarios are planned: connection plate positioning, simplified pin insertion, and sequential two-component assembly. Each scenario is evaluated at fixed-base, in-place whole-body, and building-scale mobile-manipulation levels when the preceding level satisfies the stop criteria.

Figure 6 and table 3 specify the intended evidence before the final simulation images and held-out partition counts are available. This prevents later selection of only visually successful tasks.

6.2. Fault-injection benchmark

Faults cover semantic identity, geometry and coordinates, physical properties, assembly interfaces, construction states, and task references. Every injected fault has a known location, severity, valid repair, and expected task consequence. The current CPU suite comprises 30 single faults and five compound

faults. Development, test, and challenge partitions are frozen before the agent comparison. The challenge partition includes faults that should be rejected or escalated rather than repaired automatically. Consequential findings are recorded separately from the injected root cause; for example, an unresolved robot actor also prevents capability verification.

6.3. Baselines and ablations

The baselines are direct IFC geometry conversion, frozen deterministic conversion with fixed repair, and unconstrained agentic repair. The full method combines the scene contract, multi-scale compiler, deterministic validator, provenance, and constrained agent selection from a whitelisted tool registry. All methods receive the same scene and fault, and none may inspect the pristine target scene or fault label during evaluation. Ablations remove provenance, task-level validation, task-activated payloads, typed diagnostics, or agentic tool selection one at a time.

6.4. Metrics and statistical tests

Static evaluation measures semantic preservation, relation F1, critical-fault precision/recall/F1, repair success, incorrect repair, human escalation, and runtime. Scene efficiency measures load time, peak memory, active prim count, and collision-query cost. Task evaluation measures reachability, collision-free planning, grasping, alignment, insertion, and complete-task success.

Paired scenes are compared using McNemar tests for binary outcomes and Wilcoxon signed-rank tests for time and iteration

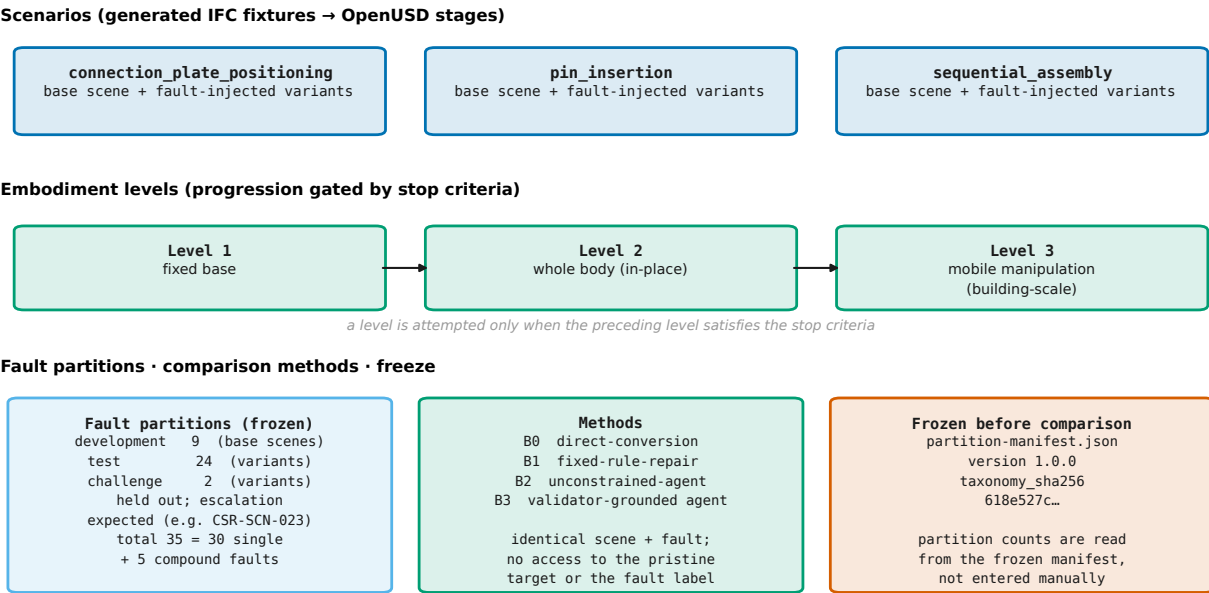


Figure 6: Benchmark organization across construction scenarios, embodiment levels, controlled scene defects, and comparison methods.

counts. Bootstrap confidence intervals quantify uncertainty. Logistic regression tests whether readiness scores predict task success while controlling for scenario and task level. Language-model methods are repeated with at least five seeds per case, with model version, decoding parameters, token use, and cost retained in the experiment registry.

Table 3: Benchmark composition. Faulted-case counts are read from the frozen partition manifest (extttdata/partition-manifest.json): 9 development, 24 test, and 2 challenge faults per scenario scene, with test and challenge faults evaluated on held-out geometric variants.

Scenario	Parameterization	Robot task levels	Faulted cases	Primary outcome
Connection-plate positioning	Component size, pose, target height, work-zone obstacle	Fixed base; whole body; mobile	9 dev / 24 test / 2 challenge	Placement pose and state completion
Simplified pin insertion	Interface axis, clearance, friction, initial misalignment	Fixed base; whole body; mobile	9 dev / 24 test / 2 challenge	Insertion depth without jam
Sequential assembly	Component order, temporary support, path obstruction, state transition	Fixed base; whole body; mobile	9 dev / 24 test / 2 challenge	Complete legal task sequence

Table 4: Frozen comparison and ablation design. All methods receive identical faulted scenes and are evaluated by the same post-repair validator and runtime tasks.

Method	Construction IR	Multi-scale USD	Deterministic scene rules	Provenance	Repair
Direct conversion	–	–	–	–	–
Fixed-rule conversion	–	partial	–	partial	fixed
Unconstrained agent	partial	partial	–	partial	agent
Full method	✓	✓	✓	✓	constrained agent

7. Results

7.1. CPU implementation check

The current local run is reported only as a software-verification milestone. Across three source scenes and 35 fault variants per scene, the benchmark created 105 cases containing 141 expected rule findings. The validator recovered all expected findings without an additional finding under the registered ground truth. The fixed whitelisted-tool implementation returned 102 cases to a valid state and escalated the three unsupported-friction cases.

Validation and repair were sub-millisecond in this small CPU check, but those timings vary with host load and are retained in the run report rather than treated as a performance result.

The same implementation check compared the task-activated root with an all-loaded root over three-zone fixtures. Task activation reduced composed prim count by 20–25%, rigid-body and collider count by 40–50%, and bytes in used USD layers by 18.0–24.5%. These are structural consequences of

the fixture and confirm that the two composition conditions differ as intended; they are not substitutes for RTX load-time, memory, and collision-query measurements.

These values should not be interpreted as evidence that the method generalizes to unseen construction scenes. The fault operators were written against the same explicit contract as the validator, and the fixed repair implementation had access to trusted source evidence. Their purpose is to verify rule coverage, compound-fault accounting, audit generation, and the distinction between safe repair and escalation before the held-out and language-model comparisons are executed.

7.2. Compiler fidelity

On the three-scenario fixture suite, the IFC-to-scene compiler preserves the source semantics exactly: component identity recall, IFC GUID accuracy, categorical attribute accuracy, interface-relation and task-binding F1 are all 1.000, with zero position, dimension, and relative-mass deviation (generated/local-pipeline/reports/semantic-preservation.json). These are identity results on fixtures whose ground truth is generated jointly with the IFC source; they verify that the compiler is lossless on the contract’s semantic vocabulary, and they calibrate the fault-injection results that follow—every detection miss reported below is a property of the method under test, not of information lost in compilation.

7.3. Humanoid task outcomes under scene-model feature defects

The runtime executor for the defect-consequence study is a frozen reinforcement-learning insertion policy on the **Isaac-Assembly-G1** task: a Unitree G1 with a locked lower body transports a virtual-grasped M20 bolt and inserts it along the horizontal axis into the lower-right hole of a 2×2 bolt group on the beam end plate. Success follows the same criterion as the training reward: transverse alignment error below 12 mm and insertion depth of at least 20 mm from the plate surface. The checkpoint used (model_10249) achieves 39.0% success over 200 evaluation episodes (38.1% over 1000 episodes) without injected defects, consistent with its previously measured 35.5% baseline under an unfixed seed.

Defects from the six feature families of Section 6 were injected at episode reset as controlled deviations between the scene model observed by the policy and the ground-truth insertion target, at graded severities. All conditions share one frozen policy, one seed partition, and an identical episode-randomization stream: the injection events consume the same amount of randomness whether or not under any defect condition is the exact paired replicate of episode k under the defect-free baseline. Paired binary outcomes are compared with exact two-sided McNemar tests; b/c denote episodes that changed from success to failure and from failure to success, respectively.

Table 5 yields a clear criticality ordering. Symbolic and referential features—task references, construction states, and semantic

Table 5: Insertion success rate under injected scene-model feature defects (200 paired episodes per condition, seed 42; geometric conditions repeated with 1000 paired episodes). Δpp is the change relative to the defect-free baseline of the same batch. Significance: exact McNemar test, *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

Feature family	Injected defect	Success	Δpp	b/c	p
—	none (baseline)	39.0%	—	—	—
Geometry, hole position	± 2 mm, random direction	38.5%	−0.5	1/0	n.s.
Geometry, hole position	± 5 mm	37.0%	−2.0	4/0	n.s.
Geometry, hole position	± 10 mm	33.5%	−5.5	11/0	***
Geometry, member pose	whole node +10 mm	36.5%	−2.5	16/11	n.s.
Assembly interface	radial clearance 0.75 mm	5.0%	−34.0	68/0	***
Assembly interface	radial clearance 0.5 mm	4.0%	−35.0	70/0	***
Assembly interface	radial clearance 0.25 mm	2.0%	−37.0	74/0	***
Semantic identity	label missing (nominal prior only)	6.5%	−32.5	67/2	***
Semantic identity	label wrong (plate centre, no hole)	0.0%	−39.0	78/0	***
Task reference	wrong hole of the 2×2 group	0.0%	−39.0	78/0	***
Construction state	member not yet in place	0.0%	−39.0	78/0	***
Physical properties	bolt mass $\times 0.5$ / $\times 2$	39.0%	+0.0	0/0	n.s.

identity—are discretely fatal: a single wrong reference drives success to zero because the executor’s target is defined by them. Among continuous features, the assembly interface is the most sensitive: shrinking the effective radial clearance of the nominal $\text{\O}22$ mm hole from 1 mm to 0.75 mm already collapses success from 39% to 5%, and the diagnostic per-episode record shows the clearance gate truncating mean insertion travel from ≈ 50 mm to below 5 mm, reproducing the physical jam-

ming mechanism rather than an artefact of the success metric.

The geometric family separates two effects that are easily conflated. A 10 mm error in the *absolute* pose of the whole connection node is almost entirely compensated by the policy’s closed-loop perception (−2.5 pp, $b/c=16/11$, n.s.), which was trained with ± 5 cm domain randomization of the node pose. The same 10 mm applied to the hole position *relative to the observed member*—the

quantity the scene model actually certifies and the policy cannot see—reduces success significantly (-5.5 pp, $p < 0.001$). Repeating the geometric conditions with 1000 paired episodes sharpens this gradient: 2 mm ~~remains~~ remains non-significant (37.5% vs. 38.1% baseline, $p = 0.109$), 5 mm becomes significant (36.8%, $p = 0.015$), and 10 mm is strongly significant (31.3%, $p = 5.4 \times 10^{-16}$), while the whole-member shift remains non-significant (35.8%, $p = 0.064$). What matters for task success is therefore not the accuracy of absolute coordinates but the accuracy of coordinates *relative to the perceptual anchor* used by the controller.

Physical-property defects (bolt mass $\times 0.5$ and $\times 2$) produced bit-identical outcomes ($b/c=0/0$). This null result is structural, not statistical: the executor’s virtual-grasp scaffold makes the bolt kinematic, so mass and friction have no dynamical channel in this task stage. We therefore report the physical family as *not consequential for the transport-and-insert motion chain* and defer its force-level criticality to a full-contact ablation (Section 8).

All findings replicate under a second seed partition (seed 123, 200 paired episodes per condition): success rates deviate by at most 2.5 pp from the seed-42 values, and every condition that was significant remains significant with the same direction and comparable magnitude, while every non-significant condition remains non-significant (Section 8). Under seed 123 the 5 mm hole-position defect is significant already at 200 episodes (35.0% vs. 40.0% baseline, $p = 0.002$), corroborating the power-upgraded seed-42 gradient.

7.4. Detection and repair on the frozen partition

Table 6 reports the registered run over the frozen partition of Section 6 (105 cases: 9 development, 24 test, and 2 challenge faults over three scenario scenes, with test and challenge faults evaluated on held-out geometric variants; manifest `data/partition-manifest.json`, run B-CPU-DETREp-0001). Direct conversion detects none of the scene-level construction faults: its generic structural check cannot name a single rule finding, and it attempts no repair. The frozen rule set with one whitelisted repair action per rule recovers every expected finding on all 105 cases (precision and recall 1.000, no false positives), returns 102 cases to a state equivalent to the pristine scene (97.1%, bootstrap 95% CI [93.3, 100]), and escalates exactly the three challenge cases whose friction record lacks an authoritative source (2.9%)—the designed behaviour, since fabricating a physical parameter would be an unsupported repair. No repair that passed revalidation deviated from the pristine scene (incorrect repair 0.0%). The detection gap between the two methods is significant on the paired cases (exact McNemar, $b=105$, $c=0$, $p < 10^{-31}$). The language-model rows were run against a frozen self-hosted instruction model (Qwen2.5-Coder-14B-Instruct-AWQ, 5 seeds per case, 525 episodes per method, run B-CPU-LLM-0001). The contrast is stark. Left unconstrained with the same scene and a prose problem report, the agent returns a scene equivalent to the pristine one in only 32.8% of episodes; 22.7% of its repairs pass de-

terministic revalidation while deviating from the pristine scene—invented identifiers, invented preconditions, and invented physical values that no rule can distinguish from real ones—and it escalates 38.3% of cases, including repairable faults, while 6.3% of replies are unusable outright (mostly truncation at the frozen decoding budget). The validator-grounded agent, restricted to selecting among the same whitelisted tools as the fixed-rule row, matches the deterministic ceiling: 96.8% valid repair versus 97.1% (paired McNemar, $b=2$, $c=0$, $p = 0.5$), with zero incorrect repairs across all 525 episodes. The contract therefore converts an otherwise unsafe agent into a safe one without measurable capability loss (fixed-rule vs free-form: $p = 3.6 \times 10^{-102}$); the fixed-rule row is the deterministic core that the full method’s agent is restricted to select from.

A scale sweep over synthetic floors with 1–20 work zones (four members each) confirms that the composition mechanism, not the fixture size, drives the saving: the task-activated working set grows only with the global layer plus the single active zone (15→91 prims), while the all-loaded composition grows linearly with the zone count (15→205 prims); at 20 zones, task activation touches 56% fewer prims and 55% fewer layer bytes (`resource_efficiency.csv`, run `A3-CPU-RES-0001`).

7.5. External IFC sanity check

As a check outside the generated fixtures, we compiled the buildingsMART IFC 4.3 ADD2 reference model `Building-Architecture.ifc` (13 elements,

2 spaces, no task entities; `data/external/`). The compiler produced an empty task scene by design—elements without CSR property-set annotations are skipped rather than guessed—and the validator refused readiness with two critical findings (no mass or tolerance evidence records). Unannotated real deliveries are therefore compilable but never silently treated as task-ready; the CSR annotation layer is the explicit bridge, and its absence is itself a contract finding (`generated/external-ifc/report.json`).

7.6. Readiness scores and the boundary of contract visibility

To test whether the contract’s readiness signal predicts downstream outcomes (hypothesis H4), each humanoid defect condition of Section 7.3 was re-expressed as its scene-contract equivalent on the matching fixture scenario and scored by the validator (readiness = fraction of the 30 rules not violated). Readiness and measured success correlate positively (Spearman $\rho = 0.54$ across conditions), but the informative analysis is the confusion structure. Every condition the validator flags was catastrophic at runtime (semantic identity, task reference, construction state: success $\leq 6.5\%$), and the validator never flags a non-consequential condition (specificity 100%). However, four consequential condition types pass the contract silently: hole-coordinate errors of 5–10 mm and all three reduced-clearance conditions. Contract visibility therefore ends exactly at the boundary of *declared* tolerances: the validator checks that tolerances and references exist and are well-formed, but not that the

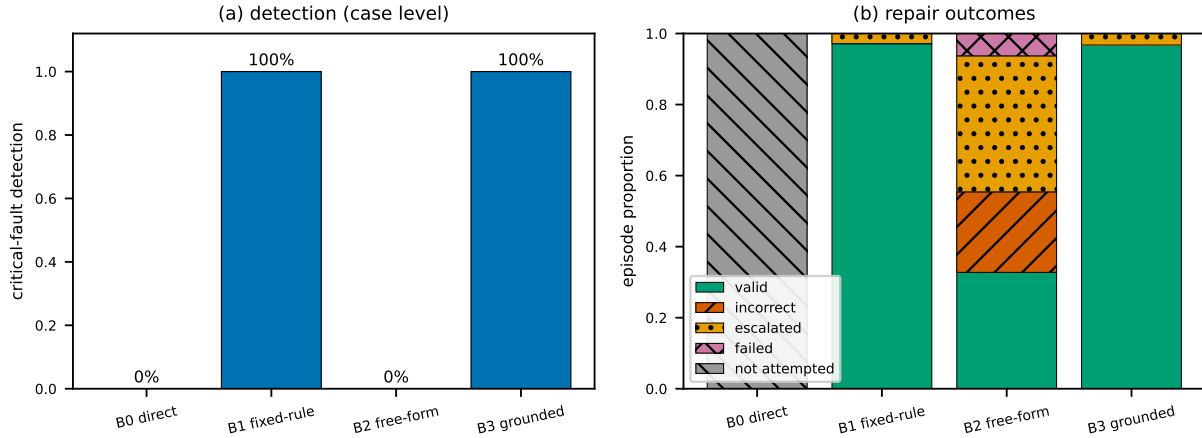


Figure 7: Detection and repair performance across the frozen partition (105 cases; agent methods 5 seeds per case). (a) Case-level critical-fault detection by method; the free-form agent produces no typed findings (n/a, see text). (b) Repair-outcome composition per method. The validator-grounded agent matches the deterministic ceiling with zero incorrect repairs; the unconstrained agent’s incorrect and over-escalation shares are the unsafe modes the contract eliminates. Colour-blind-safe palette with redundant hatching; generated from the registered result files by `scripts/make_figures.py`.

coordinates inside them are faithful. We register two candidate rules to close this gap: member-relative interface coordinates shall match the evidence model within the declared tolerance (CSR-SCN-031), and interface clearance shall exceed the fastener envelope by the task-required minimum (CSR-SCN-032). Their precision cannot be estimated from the present fixture suite, which contains no such faults by construction—a benchmark extension motivated directly by the humanoid measurement.

Table 6: Primary detection and repair outcomes over the frozen partition (105 cases; agent methods: 5 seeds per case, 525 episodes; bootstrap 95% CIs in brackets). Rule-level precision and recall are computed over 141 expected findings per method. Direct conversion has no scene-rule detector, so its detection cells are not applicable. Time is the mean validation-plus-repair time per case (agent rows: mean episode wall time including model inference). ^b The free-form agent produces no typed findings; its observable is repair, scored by independent deterministic revalidation. ^c The full method receives typed validator findings as input; detection equals the fixed-rule row by construction.

Method	Precision	Recall	Critical recall	Valid repair	Incorrect repair	Escalation	Time
Direct conversion	n/a	n/a	n/a	n/a	n/a	n/a	<0.01 ms
Fixed-rule conversion	1.000	1.000	1.000	97.1% [93.3, 100]	0.0%	2.9%	0.31 ms
Free-form agent	n/a ^b	n/a ^b	n/a ^b	32.8% [28.8, 36.8]	22.7%	38.3%	20.0 s
Full method	1.000 ^c	1.000 ^c	1.000 ^c	96.8% [95.2, 98.1]	0.0%	3.2%	1.5 s

Table 7: Runtime resource use and task-level outcomes. Resource cells are reported per composition mode on the held-out variant scenes (structural metrics are exact; load time is the median of five cold opens and varies with host). Peak memory stayed below the 1 MB measurement threshold. Task stages are measured with the frozen RL executor on the defect-free scene (two seed partitions); planning, grasp, and alignment are not instrumented separately under the virtual-grasp scaffold (Section 7.3).

Condition	Active prims	Peak memory	Load time	Planning	Grasp	Alignment	Insertion	Full task
Direct/all loaded	28–30	<1 MB	1.4 ms	n/a	n/a	n/a	n/a	n/a
Fixed rule	identical compiler output as full method				n/a	n/a	n/a	n/a
Free-form agent	method-independent compilation output; task cells not instrumented (Section 7.3)							
Full/task activated	22–24	<1 MB	1.5 ms	n/a	n/a	n/a	39.0%/40.0%	= insertion

Table 8: Ablation and sensitivity analysis relative to the fixed-rule row of Table 6 (105 paired cases each; bootstrap 95% CIs in brackets). Task success refers to the measured humanoid outcome under the defect family corresponding to the removed information (Section 7.3): provenance/physics defects were non-consequential, while task-reference defects were universally fatal.

Ablation	Removed information or control	Critical recall	Valid repair	Resource use	Task success
No provenance	Source authority and uncertainty	0.894 [0.839, 0.941]	85.7% [79.0, 92.4]	unchanged	39.0% (=baseline)
No task validator	Task binding and action preconditions	0.787 [0.706, 0.868]	80.0% [72.4, 87.6]	unchanged	0.0%
All layers loaded	Task-triggered payload activation	1.000	97.1%	+27% prims, +20% layer bytes	n/a
Fixed repair order	Agentic tool selection	1.000	97.1%	unchanged	identical to fixed rule
Free-form diagnostics	Typed diagnostic schema	1.000 (given)	89.0% [86.3, 91.6]	unchanged	n/a

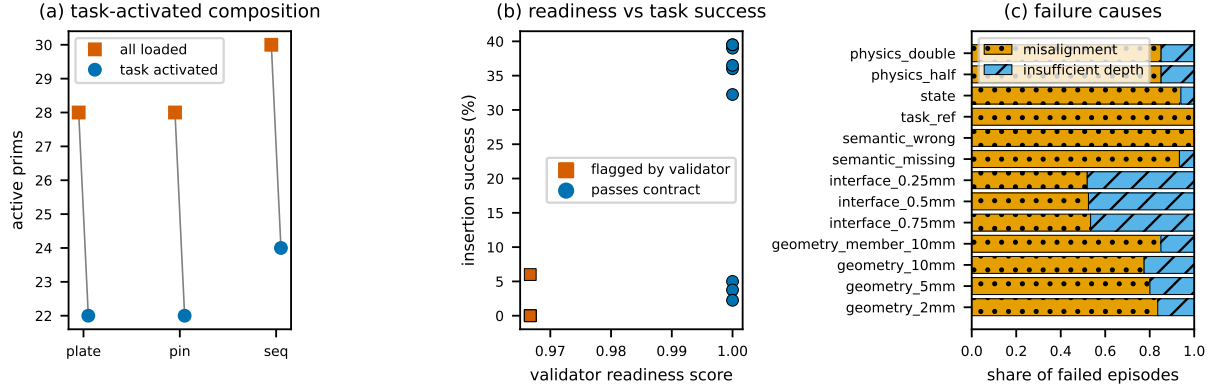


Figure 8: Resource effect of task-activated composition and the relationship between scene readiness and downstream humanoid assembly outcomes. (a) Active prim count per held-out scene, all-loaded versus task-activated composition. (b) Validator readiness score versus measured insertion success for the 14 defect conditions (two seed partitions each); the contract flags every catastrophic condition (squares, success $\leq 6.5\%$) but passes the sub-tolerance geometric and interface conditions that still degrade the task. (c) Failure-cause composition per defect condition from the per-episode ledger. Generated from `resource_efficiency.csv`, `task_outcomes.csv`, and the readiness-link report by `scripts/make_figures.py`.

(a) faulted evidence record (friction)

```
{
  "entity": "default_steel_contact",
  "property": "friction",
  "source": {
    "benchmark_protocol": "steel_contact_v0.1",
    "source_type": "unverified_model_generation",
    "value": 0.5,
    "unit": "dimensionless",
    "confidence": 0.2,
    "uncertainty_interval": [
      0.3,
      0.7
    ]
  },
  "authority_level": 1,
  "auto_repair_allowed": false,
  "requires_human_confirmation_for_deployment": true
}
```

(b) typed validator finding (CSR-SCN-023)

```
{
  "rule_id": "CSR-SCN-023",
  "severity": "critical",
  "path": "$.provenance[7].source_type",
  "message": "A critical physical property cannot rely solely on an unverified generated value."
}
```

(c) validator-grounded agent escalation

```
{
  "method": "validator-grounded-agent",
  "case": "connection_plate_positioning_v001_unsupported_generated_friction",
  "outcome": "escalated",
  "episodes_escalated": "15/15 (3 scenes x 5 seeds)",
  "note": "no authoritative friction source; repair refused rather than invented"
}
```

Figure 9: Auditable failure–diagnosis–escalation sequence for the held-out challenge case `unsupported_generated_friction (pin_insertion_v001)`. (a) The faulted provenance record: friction entered by unverified model generation with low confidence and repair blocked by policy. (b) The deterministic typed finding (CSR-SCN-023). (c) The frozen validator-grounded model escalates in all 15 episodes rather than inventing a friction value. All panels render committed artefacts (scene JSON, validation report, episode ledger) via `scripts/make_case_study_figure.py`; simulator execution frames remain pending (Section 8).

8.1. Which scene-model features are task-critical

1145 The paired defect-injection results yield a criticality ordering that is not aligned with the effort distribution of current simulation-readiness specifications, which emphasize asset-level geometry and physical fidelity.

1150 *Referential integrity is discrete-fatal..* A single wrong task reference, a stale construction state, or a wrong semantic label drove task success from 39% to exactly zero under both seed partitions. These features define *what* the executor is asked to do; no amount of perceptual or control robustness can compensate for executing the wrong target. Scene contracts should therefore treat reference closure—every task reference resolves to an existing, correctly labelled, in-place member—as a hard acceptance gate rather than a soft quality score. Notably, a *missing* semantic label is far less harmful than a *wrong* one (6.5% vs. 0.0%): the executor’s nominal-pose prior occasionally suffices, so completeness requirements can be graded while correctness requirements cannot.

The discussion separates three claims: whether the contract detects construction-specific scene defects, whether constrained repair resolves those defects reliably, and
 1135 whether improved readiness transfers to ~~run-~~ time task outcomes. This separation prevents successful simulation videos from being treated as evidence that every scene requirement is correct. The defect-consequence study of Section 7.3 speaks to the third claim
 1140 and, by inversion, to which scene-model features the first two claims must protect.

Assembly-interface tolerance is the most sensitive continuous feature.. Reducing the effective radial clearance of a nominal Ø22 mm hole from 1 mm to 0.75 mm—an error well within what manual modelling or careless unit handling can introduce—collapsed success from 39% to 5%. The per-episode diagnostics show the defect acting through the intended physical channel (insertion travel truncated from ≈ 50 mm to below 5 mm by

the clearance gate), so this is a genuine jamming phenomenon rather than a metric artifact. Interface dimensions therefore require certified values with provenance; an interface field without provenance should be treated as absent.

Geometric accuracy matters relative to the perceptual anchor, not in absolute terms.. A 10 mm shift of the entire connection node was almost fully compensated by closed-loop perception (-2.5 pp, n.s.), whereas the same 10 mm between the certified hole position and the observed member cost 5.5 pp ($p < 0.001$). The scene model’s coordinate guarantees should accordingly be specified and verified *relative to the features the controller can observe*, since absolute-frame accuracy is partially self-correcting at runtime while anchor-relative error is invisible to the controller by construction.

Physical-property criticality is stage-dependent.. Bolt mass perturbations of $\times 0.5$ and $\times 2$ produced bit-identical outcomes because the transport-and-insert executor holds the bolt in a kinematic virtual grasp. We read this as a structural property of the motion stage, not evidence that physical fidelity is unimportant: the same defect is expected to become consequential in any stage with forceful contact, and a full-contact ablation with a physically grasped bolt is required before the physical family can be ranked.

8.2. Sensitivity and robustness of the findings

Three analyses bound the statistical fragility of these conclusions.

First, all fourteen conditions were re-run under an independent seed partition (seed 123, 200 paired episodes per condition). Success rates deviate by at most 2.5 pp from the seed-42 values; every condition that was significant remains significant with the same direction and comparable magnitude, and every non-significant condition remains non-significant. The criticality ordering is therefore a property of the scene-model features, not of a particular random draw of episodes.

Second, the severity gradients are monotone where the defect acts through a continuous channel. Hole-position error at $n = 1000$ paired episodes gives 37.5%, 36.8%, and 31.3% success for 2, 5, and 10 mm against a 38.1% baseline; interface clearance gives 5.0%, 4.0%, and 2.5–2.0% for 0.75, 0.5, and 0.25 mm across the two seeds. Monotone dose-response supports a causal interpretation of the injected defect rather than threshold noise in the success criterion.

Third, the power upgrade from 200 to 1000 paired episodes changed exactly one conclusion and clarifies its meaning. At $n = 200$ the 2 mm and 5 mm hole-position defects were underpowered ($b=1$ and 4 discordant pairs); at $n = 1000$, 5 mm is significant ($p = 0.015$, corroborated at $n = 200$ under seed 123 with $p = 0.002$) while 2 mm remains non-significant ($p = 0.109$). For this executor, hole-position errors below roughly 5 mm lie inside the policy’s tolerance band, and claims about sub-5 mm criticality would require either a stricter executor or substantially larger samples. We accordingly report effect sizes alongside significance and do not rank features on p -values alone.

8.3. Boundary of pure-simulation conclusions

1290

1255 All runtime evidence in this study is simulated, and the conclusions should be read as structural rather than numerical. Rigid-body simplification, approximate contact parameters, and simplified pins in place of threaded fasteners mean that absolute success rates and clearance thresholds will shift on hardware; what transfers is the ordering and the causal channel (identification of *which* defect classes are fatal, graded, or self-correcting). Three further boundaries apply. (i) A single robot-controller combination was used: the compensation of whole-node pose error depends on the executor’s ± 5 cm pose domain randomization during training, and an executor without such randomization would likely show a larger member-pose effect. (ii) The physical-family null result is an artefact of the virtual-grasp scaffold and bounds only the transport-and-insert stage. (iii) The success criterion (12 mm alignment, 20 mm insertion depth) matches the training reward, and stricter assembly tolerances would shift all conditions downward but preserve pairing. A physical-robot validation study is deferred to future work, and the present results should be treated as lower bounds on defect consequence: defects that are benign here are not certified benign on hardware, while defects that are fatal here are fatal by construction of the task definition.

8.4. Implications for cross-runtime specifications

1320

1285 The defect-consequence profile suggests which validation rules belong in a runtime-

agnostic scene contract and which require per-runtime calibration. Reference closure, semantic-identity correctness, and construction-state consistency are fatal regardless of executor and should be specified once, as cross-runtime rules with hard-fail semantics. Interface-tolerance and anchor-relative-geometric rules have executor-dependent thresholds (here ≈ 1 mm clearance and ≈ 5 mm anchor-relative position) and should be specified parametrically, with the threshold supplied by the target controller’s tolerance band and verified by the kind of paired injection benchmark used here. Asset-level physical fidelity, finally, should be required only for task stages with a dynamical channel; mandating it uniformly imposes authoring cost without measurable task benefit.

9. Conclusion

This paper frames robot-ready construction simulation as a scene compilation and validation problem rather than a file-conversion problem. It proposes a multi-scale IFC-to-OpenUSD pipeline, a construction task-scene contract, and a deterministically grounded repair process. On a frozen fault-injection partition the contract validator recovered every expected finding without false positives and repaired 97.1% of cases to pristine-scene equivalence, escalating exactly those cases whose repair would exceed the available engineering evidence. Grounding a language-model agent in the contract preserved this reliability (96.8% valid, zero incorrect repairs), whereas the same model with-

1325 out the contract produced validator-passing
but incorrect scenes in nearly a quarter of
episodes—direct evidence that scene-level en-
1330 gineering authority, not model capability, is
the binding constraint on agentic scene re-
pair.

The humanoid defect benchmark closes the
loop between scene models and task out-
comes. Measured over 5,600 paired inser-
1335 tion episodes, defects in symbolic and refer-
ential features—semantic identity, task ref-
erences, construction states—were discretely
fatal; assembly-interface clearance was the
most sensitive continuous property; and ab-
1340 solute member-pose errors were largely ab-
sorbed by closed-loop perception. Cross-
checking these outcomes against the validator
showed perfect specificity but 44% sensitiv-
ity: the contract sees every catastrophic de-
1345 fect family yet remains blind to sub-tolerance
geometric and interface fidelity, which moti-
vates the two candidate rules registered in
this work and, more broadly, positions run-
1375 time task measurement as a calibration in-
strument for scene-readiness contracts.

1350 The results are bounded by a single execu-
tor, a single assembly task, a kinematic grasp
scaffold, and simulation-only evidence; ex-
tending the defect benchmark to contact-rich
stages and additional embodiments, and com-
1355 pleting the whole-body task levels, is the nat-
ural next step. The framework’s artefacts—
the frozen partition, registered scripts, and
per-episode ledgers—are intended to make ev-
ery number in this paper regenerable. 1385

CRedit authorship contribution state- ment

*Complete after the author list and contri-
butions are frozen.*

Declaration of competing interest

The authors declare that they have no
known competing financial interests or per-
sonal relationships that could have appeared
to influence the work reported in this paper.

Funding

*Provide funder names and grant numbers,
or state that no specific funding was re-
ceived. Keep identifying information out of
an anonymized review file if required by the
current journal instructions.*

Data and code availability

*Provide the permanent repository and
dataset DOI after the benchmark release is
archived.*

Declaration of generative AI and AI- assisted technologies

*At submission, use Elsevier’s current re-
quired wording and disclose the tools used for
coding or language support. Human authors
remain responsible for all methods, evidence,
and conclusions.*

References

1420

- 1390 [1] S. Kim, M. Peavy, P.-C. Huang, K. Kim, Development of BIM-integrated construction robot task planning and simulation system, *Automation in Construction* 127 (2021) 103720. doi:10.1016/j.autcon.2021.103720.
- 1395 [2] A. Zhu, P. Pauwels, B. De Vries, Component-based robot prefabricated construction simulation using IFC-based building information models, *Automation in Construction* 152 (2023) 104899. doi:10.1016/j.autcon.2023.104899.
- 1400 [3] H. Oyediran, W. Turner, K. Kim, M. Barrows, Integration of 4d BIM and robot task planning: Creation and flow of construction-related information for action-level simulation of indoor wall frame installation, *Journal of Information Technology in Construction* 30 (2025) 352–374. doi:10.36680/j.itcon.2025.015.
- 1405 [4] A. Zhu, P. Pauwels, E. Torta, H. Zhang, B. De Vries, Data linking and interaction between BIM and robotic operating system (ROS) for flexible construction planning, *Automation in Construction* 163 (2024) 105426. doi:10.1016/j.autcon.2024.105426.
- 1410 [5] X. Wang, H. Yu, W. McGee, C. C. Menassa, V. R. Kamat, Enabling building information model-driven human-robot collaborative construction workflows with closed-loop digital twins, *Computers in Industry* 161 (2024) 104112. doi:10.1016/j.compind.2024.104112.
- [6] L. Xu, D. Veeramani, Z. Zhu, Automated physics-based modeling of construction equipment through data fusion, *Automation in Construction* 168 (2024) 105880. doi:10.1016/j.autcon.2024.105880.
- [7] Alliance for OpenUSD, Openusd documentation, <https://openusd.org/release/>, accessed 26 July 2026 (2026).
- [8] G. Ren, C. Huang, T. Su, From building deliverables to open scene description: A pipeline for lifecycle 3d interoperability, *Buildings* 15 (24) (2025) 4503. doi:10.3390/buildings15244503.
- [9] NVIDIA Corporation, The simready standardization workflow, https://docs.omniverse.nvidia.com/simready/latest/simready-standards/standardization_workflow.html, accessed 26 July 2026 (2026).
- [10] NVIDIA Corporation, Simready faq, <https://docs.omniverse.nvidia.com/simready/latest/simready-faq.html>, accessed 26 July 2026 (2026).
- [11] X. Ye, L. Liu, M. König, Digital twin framework for humanoid robotics in construction: A conceptual approach for scene and task simulation, in: *Proceedings of the 43rd International Symposium on Automation and Robotics in*

Construction, 2026, pp. 438–445. doi:
10.22260/ISARC2026/0057.

- [12] P. Vähä, T. Heikkilä, P. Kilpeläinen, M. Järviluoma, E. Gambao, Extending automation of building construction: Survey on potential sensor technologies and robotic applications, *Automation in Construction* 36 (2013) 168–178. doi:10.1016/j.autcon.2013.08.002.
- [13] T. Bock, The future of construction automation: Technological disruption and the upcoming ubiquity of robotics, *Automation in Construction* 59 (2015) 113–121. doi:10.1016/j.autcon.2015.07.022.
- [14] H. Ardiny, S. Witwicki, F. Mondada, Construction automation with autonomous mobile robots: A review, in: 2015 3rd RSI International Conference on Robotics and Mechatronics, 2015, pp. 418–424. doi:10.1109/ICRoM.2015.7367821.
- [15] J. M. Davila Delgado, L. Oyedele, A. Ajayi, L. Akanbi, O. Akinade, M. Bilal, H. Owolabi, Robotics and automated systems in construction: Understanding industry-specific challenges for adoption, *Journal of Building Engineering* 26 (2019) 100868. doi:10.1016/j.jobe.2019.100868.
- [16] N. Melenbrink, J. Werfel, A. Menges, On-site autonomous construction robots: Towards unsupervised building, *Automation in Construction* 119 (2020) 103312. doi:10.1016/j.autcon.2020.103312.
- [17] B. Xiao, C. Chen, X. Yin, Recent advancements of robotics in construction, *Automation in Construction* 144 (2022) 104591. doi:10.1016/j.autcon.2022.104591.
- [18] K. H. Petersen, N. Napp, R. Stuart-Smith, D. Rus, M. Kovac, A review of collective robotic construction, *Science Robotics* 4 (28) (2019). doi:10.1126/scirobotics.aau8479.
- [19] Y. Xu, C. Cheung, M. S. Ng, A. Y. Kaltungo, T. Ishizawa, K. Fujimoto, Construction robotics: A systematic review of robot types, applications and human robot collaborations, *Results in Engineering* 30 (2026) 110406. doi:10.1016/j.rineng.2026.110406.
- [20] M. Afaq, S. Dutta, A. H. Gourabpasi, F. Jalaei, R. Ahmad, Robotics and AI for prefabrication of steel building components: Systematic review and future roadmap, *Automation in Construction* 190 (2026) 107107. doi:10.1016/j.autcon.2026.107107.
- [21] J. Zhang, H. Luo, J. Xu, Towards fully BIM-enabled building automation and robotics: A perspective of lifecycle information flow, *Computers in Industry* 135 (2022) 103570. doi:10.1016/j.compind.2021.103570.
- [22] W. Anane, I. Iordanova, C. Ouellet-Plamondon, Building information mod-

eling (BIM) and robotic manufacturing technological interoperability in construction: A cyclic systematic literature review, *Digital Manufacturing Technology* (2023) 1–29. doi:10.37256/dmt.3120231856.

[23] H. Yue, Q. Wang, Z. Zhao, S. Lai, G. Huang, Interactions between BIM and robotics: Towards intelligent construction engineering and management, *Computers in Industry* 169 (2025) 104299. doi:10.1016/j.compind.2025.104299.

[24] O. Odugu, F. Ghafari, E. Shourangiz, M. T. Khan, C. Wang, BIM-driven robotics in construction: A systematic review of integration methods, applications, and future directions, *Journal of Intelligent Construction* 3 (4) (2025) 1–17. doi:10.26599/JIC.2025.9180103.

[25] M. Dawod, S. Hanna, BIM-assisted object recognition for the on-site autonomous robotic assembly of discrete structures, *Construction Robotics* 3 (1–4) (2019) 69–81. doi:10.1007/s41693-019-00021-9.

[26] L. Ding, W. Jiang, Y. Zhou, C. Zhou, S. Liu, BIM-based task-level planning for robotic brick assembly through image-based 3d modeling, *Advanced Engineering Informatics* 43 (2020) 100993. doi:10.1016/j.aei.2019.100993.

[27] O. Wong Chong, J. Zhang, R. M. Voyles, B.-C. Min, BIM-based simulation of construction

robotics in the assembly process of wood frames, *Automation in Construction* 137 (2022) 104194. doi:10.1016/j.autcon.2022.104194.

[28] Y. Gao, J. Meng, J. Shu, Y. Liu, BIM-based task and motion planning prototype for robotic assembly of COVID-19 hospitalisation light weight structures, *Automation in Construction* 140 (2022) 104370. doi:10.1016/j.autcon.2022.104370.

[29] K. Kim, M. Peavy, BIM-based semantic building world modeling for robot task planning and execution in built environments, *Automation in Construction* 138 (2022) 104247. doi:10.1016/j.autcon.2022.104247.

[30] M. S. Moura, C. Rizzo, D. Serrano, BIM-based localization and mapping for mobile robots in construction, in: *2021 IEEE International Conference on Autonomous Robot Systems and Competitions*, 2021, pp. 12–18. doi:10.1109/ICARSC52212.2021.9429779.

[31] M. A. Gopee, S. A. Prieto, B. García de Soto, Improving autonomous robotic navigation using IFC files, *Construction Robotics* 7 (3–4) (2023) 235–251. doi:10.1007/s41693-023-00112-8.

[32] J. Chen, W. Lu, Y. Fu, Z. Dong, Automated facility inspection using robotics and BIM: A knowledge-driven approach, *Advanced Engineering Informatics* 55 (2023) 101838. doi:10.1016/j.aei.2022.101838.

- [33] X. Zhao, C. C. Cheah, A building information modeling (BIM)-enabled robotic system for automated indoor construction progress monitoring, in: 2024 IEEE International Conference on Real-time Computing and Robotics, 2024, pp. 38–43. doi:10.1109/RCAR61438.2024.10670745.
- [34] F. Bahreini, M. Nasrollahi, A. Taher, A. Hammad, Ontology for BIM-based robotic navigation and inspection tasks, Buildings 14 (8) (2024) 2274. doi:10.3390/buildings14082274.
- [35] M. Amani, R. Akhavian, Safe and trustworthy robot pathfinding with BIM, MHA*, and NLP, Construction Robotics 9 (2) (2025). doi:10.1007/s41693-025-00161-1.
- [36] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012, pp. 5026–5033. doi:10.1109/IROS.2012.6386109.
- [37] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, Isaac gym: High performance GPU-based physics simulation for robot learning, arXiv preprint arXiv:2108.10470 (2021). arXiv:2108.10470.
- [38] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2017, pp. 23–30. doi:10.1109/IROS.2017.8202133.
- [39] buildingSMART International, Industry foundation classes 4.3, https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/, accessed 26 July 2026 (2026).
- [40] IfcOpenShell Contributors, Ifcopenshell 0.8.5 documentation, <https://docs.ifcopenshell.org/>, accessed 26 July 2026 (2026).
- [41] C. Xia, K. Zhu, Z. Wang, F. Liu, Z. Zhang, Y. Duan, SimRecon: SimReady compositional scene reconstruction from real videos, arXiv preprint arXiv:2603.02133 (2026). arXiv:2603.02133. URL <https://arxiv.org/abs/2603.02133>
- [42] X. Wang, L. Liu, T. Ding, A. Choi, C. Huang, M. Zhao, Z. Li, J. Jiang, C. Yu, S. Liu, W. Xu, Z. Su, Embodied-Gen V2: An agentic, simulation-ready 3d world engine for embodied AI, arXiv preprint arXiv:2607.07459 (2026). arXiv:2607.07459. URL <https://arxiv.org/abs/2607.07459>
- [43] G. Chen, Q. Xia, J. Peng, H. Zhang, B. Ma, J. Qian, Z. Jiao, B. Zhou,

- L. Ye, K. Zhang, K. Wang, W. Zeng, Y. Chen, P. Yang, Z. Zeng, H. Wang, C. Liu, A. Yuille, F. Shi, C. Zheng, Y. Li, C. Jiang, P. Y. Chen, Agentic Real2Sim: Physics-based world modeling with vision-language agents, arXiv preprint arXiv:2607.19190 (2026). arXiv:2607.19190.
URL <https://arxiv.org/abs/2607.19190>.
- [44] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, P. Abbeel, Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation, in: Robotics: Science and Systems XX, 2024, p. 61. doi:10.15607/RSS.2024.XX.061.
- [45] C. Boje, A. Guerriero, S. Kubiak, Y. Rezgui, Towards a semantic construction digital twin: Directions for future research, Automation in Construction 114 (2020) 103179. doi:10.1016/j.autcon.2020.103179.
- [46] R. Sacks, I. Brilakis, E. Pikas, H. S. Xie, M. Girolami, Construction with digital twin information systems, Data-Centric Engineering 1 (2020) e14. doi:10.1017/dce.2020.16.
- [47] J. Tuomisto, Y. Zheng, I. Chauhan, O. Seppänen, J. Pajarinen, Automating on-site object inspection with a quadruped robot and BIM, Automation in Construction 181 (2026) 106571. doi:10.1016/j.autcon.2025.106571.
- [48] R. Akhavian, M. Amani, J. Mootz, B. Beheshti, R. Ashe, Semantic-gravity iterative closest point and AI-assisted navigation for robot-ready construction digital twins, Automation in Construction 190 (2026) 107134, article in press. doi:10.1016/j.autcon.2026.107134.
- [49] Q. Xu, A. Zhu, G. Xu, Z. Shao, J. Zhang, H. Zhang, FEM-based real-time task planning for robotic construction simulation, Automation in Construction 170 (2025) 105935. doi:10.1016/j.autcon.2024.105935.
- [50] A. Zhu, Z. Shao, X. Chai, E. Ma, Q. Li, H. Ye, H. Zhang, Component-based BIM-semantic web integration for enhanced robotic visual perception, Automation in Construction 177 (2025) 106270. doi:10.1016/j.autcon.2025.106270.
- [51] H. You, Y. Ye, T. Zhou, Q. Zhu, J. Du, Robot-enabled construction assembly with automated sequence planning based on ChatGPT: RoboGPT, Buildings 13 (7) (2023) 1772. doi:10.3390/buildings13071772.
- [52] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., Do as i can, not as i say: Grounding language in robotic affordances, in: Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, 2023, pp. 287–318.

URL <https://proceedings.mlr.press/v205/ahn23a.html>

- [53] W. Huang, F. Xia, D. Shah, D. Driess, A. Zeng, Y. Lu, P. Florence, I. Mordatch, S. Levine, K. Hausman, B. Ichter, Inner monologue: Embodied reasoning through planning with language models, in: Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, 2023, pp. 1769–1782.

URL <https://proceedings.mlr.press/v205/ahn23a.html>

- [54] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, A. Zeng, Code as policies: Language model programs for embodied control, in: 2023 IEEE International Conference on Robotics and Automation, 2023, pp. 9493–9500. doi:10.1109/ICRA48891.2023.10160591.

- [55] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, A. Garg, Progprompt: Generating situated robot task plans using large language models, in: 2023 IEEE International Conference on Robotics and Automation, 2023, pp. 11523–11530. doi:10.1109/ICRA48891.2023.10161317.

- [56] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al., PaLM-E: An embodied multimodal language model, in: Proceedings of the 40th International

Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 8469–8488.

URL <https://proceedings.mlr.press/v202/driess23a.html>

- [57] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al., RT-2: Vision-language-action models transfer web knowledge to robotic control, arXiv preprint arXiv:2307.15818 (2023). arXiv:2307.15818.

- [58] S. S. Kannan, V. L. N. Venkatesh, B.-C. Min, SMART-LLM: Smart multi-agent robot task planning using large language models, arXiv preprint arXiv:2309.10062 (2023). arXiv:2309.10062.

- [59] X. Lin, Y. Wu, H. Yang, Y. Huang, Y. Zhang, J. Ji, Y. Zhang, Improving efficiency of answer set planning with rough solutions from large language models for robotic task planning, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, 2025, pp. 4570–4578. doi:10.24963/ijcai.2025/509.

- [60] M. Frasheri, G. Lumer-Klabbers, J. L. Kvistgaard, J. O. Hausted, C. Thule, L. Esterle, P. G. Larsen, Building safer robots: From simulation to hardware deployment, in: 2023 27th International Conference on Methods and Models in Automation and Robotics, 2023, pp.

63–68. doi:10.1109/MMAR58394.2023.10242467.

[61] buildingSMART International, Information delivery specification (ids) 1.0, <https://standards.buildingsmart.org/IDS/>, accessed 13 September 2026 (2024).

[62] buildingSMART International, buildingsmart data dictionary (bsdd), <https://www.buildingsmart.org/users/services/buildingsmart-data-dictionary/>, accessed 13 September 2026 (2025).